

Object-Oriented Programming: Java Inheritance

Comprehensive Lab

Detailed technical documentation covering Multilevel, Hierarchical, Hybrid, and Multiple Inheritance with step-by-step logic, algorithms, UML architecture, source code, and outputs.

Program 1: Multilevel Inheritance (Inherit2)

Type: Multilevel

Demonstrates a linear inheritance chain where a derived class serves as the base class for another derived class.

1. Problem Statement / Question

Write a Java program to implement **Multilevel Inheritance** with classes `animal`, `bird`, and `sea`. Class `animal` defines attributes and methods for an animal; `bird` extends `animal` adding bird attributes and methods; and `sea` extends `bird` adding academic marks data. In a driver class `Inherit2`, instantiate the bottom child class (`sea`) and invoke methods inherited across all hierarchy levels.

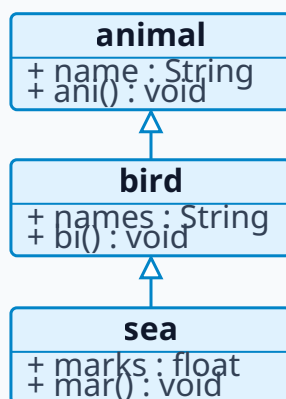
2. Step-by-Step Process

1. Define superclass `animal` with field `name` = "lion" and method `ani()`.
2. Define intermediate subclass `bird` that extends `animal`, introducing field `names` and method `bi()`.
3. Define subclass `sea` extending `bird` with field `marks` and method `mar()`.
4. In `main()`, create object as of class `sea`.
5. Invoke methods `mar()`, `bi()`, and `ani()` sequentially via object `as`.

3. Algorithm

1. **Start.**
2. Declare class `animal`: method `ani()` prints "lion is the my fav animal".
3. Declare class `bird` extends `animal`: method `bi()` prints "parrot is my fav bird".
4. Declare class `sea` extends `bird`: method `mar()` prints "you got 1 st rank".
5. In main class `Inherit2`, instantiate: `sea as = new sea();`
6. Call `as.mar()`, `as.bi()`, and `as.ani()`.
7. **Stop.**

4. UML Class Diagram



5. Java Source Code

```
class animal {
    String name = " lion";
    void ani() {
        System.out.println(" lion is the my fav animal");
    }
}

class bird extends animal {
    String names = " parrot , sparrow";
    void bi() {
        System.out.println("parrot is my fav bird"); // multilevel inheritance
    }
}

class sea extends bird {
    float marks = 99.23f;
    void mar() {
        System.out.println(" you got 1 st rank");
    }
}

public class Inherit2 {
    public static void main(String[] args) {
        // Create an object of 3rd level sub-class
        sea as = new sea();
        as.mar(); // Own method
        as.bi(); // Inherited from bird
        as.ani(); // Inherited from animal
    }
}
```

6. Sample Output

```
you got 1 st rank
parrot is my fav bird
lion is the my fav animal
```

Program 2: Hierarchical Inheritance (Inherit3)

Type: Hierarchical Demonstrates a parent base class being independently inherited by two or more sibling subclasses.

1. Problem Statement / Question

Write a Java program demonstrating **Hierarchical Inheritance** where multiple derived classes (bird and sea) both directly extend the same base class animal. Instantiate the derived class sea and access its own methods as well as the base class animal methods.

2. Step-by-Step Process

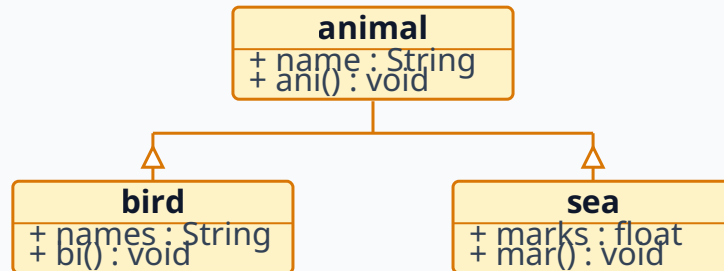
1. Create base class animal containing name and ani().
2. Derive class bird directly from animal using extends.

3. Algorithm

1. **Start.**
2. Define parent class animal with method ani().
3. Define child class 1: bird extends animal with method bi().

3. Derive another sibling class `sea` directly from `animal` using `extends`.
4. In `Inherit3` main method, instantiate sibling class `sea`.
5. Call `mar()` of class `sea`, and call `ani()` twice directly through the inherited link.
4. Define child class 2: `sea` extends `animal` with method `mar()`.
5. In driver class `Inherit3`, construct instance `sea` as `= new sea();`
6. Execute `as.mar()` and `as.ani()`.
7. **Stop.**

4. UML Class Diagram



5. Java Source Code

```
class animal {
    String name = " lion";
    void ani() {
        System.out.println(" lion is the my fav animal");
    }
}

class bird extends animal {
    String names = " parrot , sparrow";
    void bi() {
        System.out.println("parrot is my fav bird");
    }
}

class sea extends animal {
    float marks = 99.23f;
    void mar() {
        System.out.println(" you got 1 st rank");
    }
}

public class Inherit3 {
    public static void main(String[] args) {
        // Create an object for subclass extending base class directly
        sea as = new sea();
        as.mar();
        as.ani();
        as.ani();
    }
}
```

6. Sample Output

```
you got 1 st rank
lion is the my fav animal
lion is the my fav animal
```

Program 3: Hybrid Inheritance (Multilevel + Hierarchical) (Inherit4)

Type: Hybrid Combines multilevel inheritance (fir → sec) with hierarchical branching (sec → thi and sec → four).

1. Problem Statement / Question

Implement a **Hybrid Inheritance** architecture in Java consisting of four classes: base class fir, derived class sec, and two child classes thi and four that both inherit from sec. Demonstrate method invocation across various levels by creating instances of four, sec, and thi.

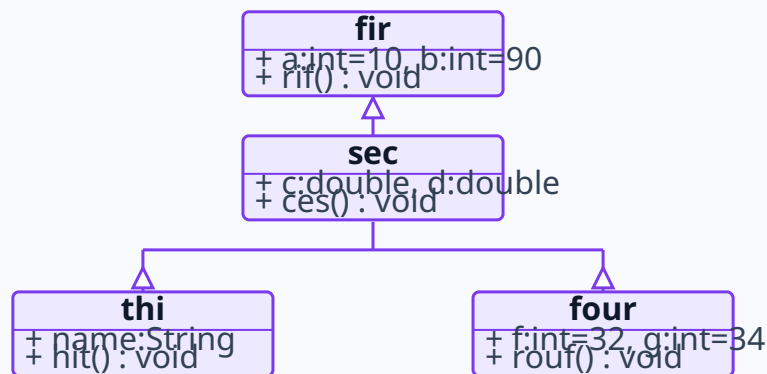
2. Step-by-Step Process

1. Create base class fir with addition method rif().
2. Create sec extending fir with floating point addition method ces().
3. Create class thi extending sec with method hit().
4. Create class four extending sec with multiplication method rouf().
5. In class Inherit4, instantiate objects oj (four), jo (sec), and joj (thi) to invoke their respective hierarchy methods.

3. Algorithm

1. **Start.**
2. Define fir with a=10, b=90 and rif() printing (a+b).
3. Define sec extends fir with ces() printing (c+d).
4. Define thi extends sec with hit() printing name.
5. Define four extends sec with rouf() printing (f*g).
6. Instantiate oj of four → execute rouf(), ces().
7. Instantiate jo of sec → execute rif(), ces().
8. Instantiate joj of thi → execute hit(), ces().
9. **Stop.**

4. UML Class Diagram



5. Java Source Code

```
class fir {
    int a = 10, b = 90;
    void rif() {
        System.out.println(" addition is " + (a + b));
    }
}

class sec extends fir {
    double c = 2.34f, d = 4.56f;
    void ces() {
        System.out.println(" th eresult is " + (c + d));
    }
}

// Multiple classes branching off sec (Hybrid Inheritance)
class thi extends sec {
    String name = "heamnth , phk";
    void hit() {
        System.out.println(" name is" + name);
    }
}

class four extends sec {
    int f = 32, g = 34;
    void rouf() {
        System.out.println(" result is " + (f * g));
    }
}

public class Inherit4 {
    public static void main(String[] args) {
        // Create object for 4th sub class
        four oj = new four();
        oj.rouf();
        oj.ces();
        oj.ces();

        // Create object for intermediate class
        sec jo = new sec();
        jo.rif();
        jo.ces();

        // Create object for sibling branch
        thi joj = new thi();
        joj.hit();
        joj.ces();
    }
}
```

6. Sample Output

```
result is 1088
th eresult is 6.8999998569488525
th eresult is 6.8999998569488525
addition is 100
th eresult is 6.8999998569488525
name isheamnth , phk
th eresult is 6.8999998569488525
```

Program 4: Multiple Inheritance via Interfaces (MultipleInherit)

Type: Multiple (Interfaces) Overcomes Java's single class inheritance constraint by implementing multiple interfaces simultaneously.

1. Problem Statement / Question

Java does not support multiple inheritance with classes due to the Diamond Problem. Write a Java program to achieve **Multiple Inheritance using Interfaces** by defining two interfaces `first` and `second`, each having constant variables and abstract methods. Implement both interfaces within class `MultipleInherit`, override their methods, and test through an instance.

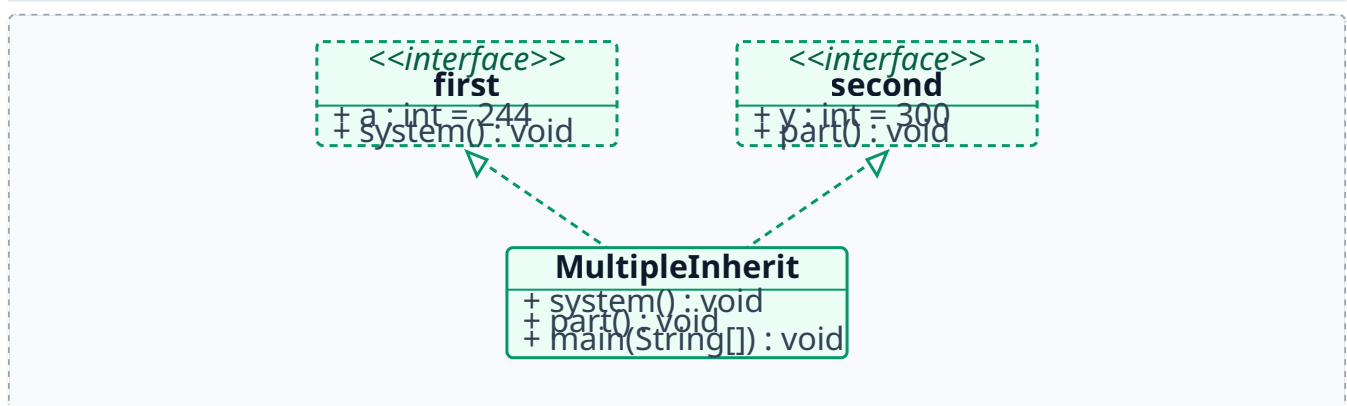
2. Step-by-Step Process

1. Declare interface `first` with constant field `a = 244` and abstract method `void system()`.
2. Declare interface `second` with constant field `y = 300` and abstract method `void part()`.
3. Create class `MultipleInherit` implementing both interfaces using `implements first, second`.
4. Provide public implementations of `system()` and `part()` with `@Override`.
5. In `main()`, instantiate `MultipleInherit` and invoke both methods.

3. Algorithm

1. **Start.**
2. Declare interface `first` → constant `a = 244`, method `system()`.
3. Declare interface `second` → constant `y = 300`, method `part()`.
4. Define class `MultipleInherit` implements `first, second`.
5. Implement `system()`: print *"the multiple inheritance value is " + a*.
6. Implement `part()`: print *"the second interface of value is " + y*.
7. Instantiate `MultipleInherit` `mi = new MultipleInherit();`
8. Call `mi.system()` and `mi.part()`.
9. **Stop.**

4. UML Class Diagram



5. Java Source Code

```
interface first {
    int a = 244;
    void system();
}

// Second interface with initialized variable and abstract method
interface second {
    int y = 300;
    void part();
}

// Inherit multiple interfaces using the 'implements' keyword
public class MultipleInherit implements first, second {
    // Override methods defined in the interfaces
    @Override
    public void system() {
        System.out.println("the multiple inheritance value is" + a);
    }

    @Override
    public void part() {
        System.out.println("the second interface of value is " + y);
    }

    public static void main(String[] args) {
        // Create instance of implementing class
        MultipleInherit mi = new MultipleInherit();
        // Call implemented methods
        mi.system();
        mi.part();
    }
}
```

6. Sample Output

```
the multiple inheritance value is244
the second interface of value is 300
```